Byte Club Low-Level Design Report

Table Of Content

1. Introduction **3**
1.1 Object design trade-offs 3
1.1.1 Latency vs. Accuracy (Gecikme ve Doğruluk Dengesi) 3
1.1.2 Granularity vs. Complexity (Ayrınıtılılık ve Karmaşıklık) 4
1.1.3 Precision vs. Robustness (Hassasiyet ve Dayanıklılık) 4
1.1.4 Visual Fidelity vs. Rendering Performance (Görsel Sadakat ve Render Performansı) 4
1.1.5 Component Decoupling vs. Synchronization (Bileşen Ayrımı ve Senkronizasyon) 4
1.1.6 Black-Box Automation vs. Explainability (Tam Otomasyon ve Açıklanabilirlik) 5
1.1.7 Completeness vs. Relevance (Kapsayıcılık ve İlgi Çekicilik) 5
1.2 Interface documentation guidelines 5
1.2.1 Data Exchange Format (Veri Değişim Formatı) 5
1.2.2 Temporal Synchronization Standard (Zaman Senkronizasyonu) 6
1.2.3 Naming Conventions (İsimlendirme Kuralları) 6
1.2.4 API Response Template (Cevap Şablonu) 6
1.2.5 Interface Versioning (Versiyonlama) 7
1.3 Engineering standards (e.g., UML and IEEE) 7
1.3.1 Modeling Standards (Modelleme Standartları) 7
1.3.2 Documentation Format (Sınıf Dokümantasyon Formatı) 7
1.3.3 Citation Standards (Referans Standartları) 7
1.4 Definitions, acronyms, and abbreviations 8
1.5 Use Cases 9
2. Packages **10**
2.1 Server 12
2.1.1 Video Analysis Package (video_analysis) 12
2.1.2 Analytics Package (analytics_engine) 13
2.1.3 VAR Package (var_system) 13
2.1.4 Content Generation Package (content_gen) 13
2.1.5 Data Access Package (data_layer) 14
2.1.6 Simulation Package (simulation_engine) 14
2.2 Client 15
2.2.1 View Package (ui) 15
2.2.2 Sync Package (sync/) 16
3. Class Interfaces **16**
3.1 Server 16
3.1.1 Video Analysis Package (video_analysis) 16
3.1.1.1 ObjectDetector 16

3.1.1.2 TeamTracker 17
3.1.1.3 FieldCalibrator 17
3.1.1.4 OCRExtractor 18
3.1.2 Tracking Package (tracking) 20
3.1.3 VAR Package (var_system) 22
3.1.3.1 VARClipIngestor 22
3.1.3.2 RuleRetriever 23
3.1.3.3 DecisionExplainer 24
3.1.3.4 EvidencePackager 27
3.1.4 Content Generation Package 29
3.1.4.1 VideoSummarizer 29
3.1.4.2 NarratorService 30
3.1.4.3 SimDataBuilder 30
3.1.5 Data Access Package (data_layer) 32
3.1.5.1 MatchRepository 32
3.1.5.2 StorageManager 33
3.1.6 Simulation Package (simulation_engine) 34
3.1.6.1 TeamDefinitionSO 34
3.1.6.2 PlayerDefinitionSO 35
3.1.6.3 MetersToWorldMapper 36
3.1.6.4 FrameSnapshotApplier 37
3.1.6.5 ShotCameraController 38
3.1.6.6 FootballSOGeneratorWindow 38
3.1.6.7 ShotGeometryConfig 39
3.1.6.8 ShotOpennessEvaluator 40
3.1.6.9 BestPassEvaluator 40
3.1.6.10 ShotOpennessRunner 41
3.1.6.11 PitchAutoAlign 41
3.1.6.12 PlayerVisual 42
3.1.6.13 ShotDebugDrawer 42
3.1.6.14 FrameSnapshotDTO 43
3.1.6.15 FieldBoundsGizmo 44
3.2 Client **46**
3.2.1 View Package (ui) 46
3.2.1.1 UploadView 46
3.2.1.2 SummaryPlayerView 46
3.2.1.3 SimulationView 47
3.2.1.4 NarrationView 47
3.2.1.5 VARAnalysisView 48
3.2.1.6 JobStatusView 48
3.2.2 Sync Package (sync) 49
3.2.2.1 SyncCoordinator 49
3.2.2.2 MasterClock 50

3.2.3 Tracking & Visualization 51
3.2.3.1 TrackingOverlayView 51
4. Glossary 52
5.Appendix 54
5.1 Traceability Matrix 54
6. References 56

1. Introduction

Bu Low-Level Design (LLD) dokümanı, futbol maç videolarının otomatik analizi ile zenginleştirilmiş içerik üreten sistemin detaylı tasarım mimarisini ve uygulama esaslarını tanımlamaktadır. Sistem; **(1)** maç videolarından önemli anların tespiti ve özetlenmesi, **(2)** takım bazlı oyuncu ve top takibi, **(3)** kritik şut anlarının 3D simülasyon ortamında görselleştirilmesi, **(4)** özet görüntülerle senkronize çalışan yapay zeka spiker anlatımı ve **(5)** bağımsız VAR klipleri için RAG tabanlı kural yorumlama yeteneklerini kapsamaktadır.

Dokümanın temel amacı; sistemin modüllerini paket seviyesinde ayrıştırmak, modüller arası veri sözleşmelerini ve servis çağrılarını standartlaştırmaktır. Ayrıca her alt sistem için sınıf hiyerarşisi, sorumluluk dağılımı ve metot seviyesinde tasarım detayları sunularak, geliştirme sürecinde ekip üyeleri arasında teknik dil birliğinin sağlanması ve olası entegrasyon risklerinin implementasyon öncesinde minimize edilmesi hedeflenmektedir.

Sistemin mimari tasarımı, verimliliği ve modülerliği artırmak amacıyla iki ana pipeline üzerinden kurgulanmıştır:

- **Maç Özeti Pipeline:** Ham maç videosunun işlenerek özet görüntü, simülasyon verisi ve spiker anlatımının üretildiği ana akış.
- **VAR Pipeline:** Özet akışından bağımsız olarak, yaklaşık 10 saniyelik tartışmalı pozisyon kliplerinin RAG mimarisi ile analiz edilip kural tabanlı açıklamaların üretildiği yan akış.

1.1 Object design trade-offs

1.1.1 Latency vs. Accuracy (Gecikme ve Doğruluk Dengesi)

Video özetleme ve olay tespiti (action spotting) için kullanılan derin öğrenme modelleri, yüksek doğruluk hedeflediğinde GPU maliyetini ve işlem süresini artırmaktadır.

- **Karar:** Projede gerçek zamanlı (real-time) analiz zorunluluğu kaldırılarak, **çevrimdışı (batch) işleme** modeli benimsenmiştir.
- **Gerekçe:** Canlı yayın gecikmesi kısıtı olmadan, daha derin ağların kullanılmasına ve sistemin hesaplama maliyetinin zamana yayılmasına olanak tanınmıştır.

1.1.2 Granularity vs. Complexity (Ayrıntılılık ve Karmaşıklık)

90 dakikalık bir maç boyunca her bir oyuncunun bireysel kimliğini (ID) korumak; kamera açısı değişimleri, oyuncu örtüşmeleri (occlusion) ve oyuncu değişiklikleri nedeniyle sistemin hata yapma olasılığını (fragility) artırmaktadır.

- **Karar:** Bireysel ID takibi yerine **Takım Bazlı Takip (Team-Based Tracking)** yöntemi seçilmiştir.
- **Gerekçe:** Projenin asıl amacı taktiksel dizilişleri ve bölgesel yoğunluğu analiz etmektir. Takım bazlı ayrım, bu amacı karşılarken sistem karmaşıklığını ve ID switch hatalarını minimize eder.

1.1.3 Precision vs. Robustness (Hassasiyet ve Dayanıklılık)

Kamera görüntüsünü kuş bakışı saha koordinatlarına (2D) çevirmek için kullanılan Homography dönüşümü, taktiksel analiz için kritik olsa da, saha çizgilerinin net görünmediği karelerde kararsız çalışabilir.

- **Karar:** Homography dönüşümü için bir **"Fallback (Geri Çekilme)" mekanizması** tasarlanmıştır.
- **Gerekçe:** Sistem, dönüşüm kalitesini (reprojection error) izler; hata oranı belirli bir eşiği aşarsa simülasyon yerine doğrudan video koordinatlarını kullanarak sistemin çökmesini (crash) veya hatalı veri üretmesini engeller.

1.1.4 Visual Fidelity vs. Rendering Performance (Görsel Sadakat ve Render Performansı)

Simülasyon ortamında (Unity) fotorealistik grafikler kullanmak, render süresini uzatır ve model karmaşıklığını artırır.

- **Karar:** Gerçekçilik yerine **"Açıklayıcılık (Explainability)"** önceliklendirilmiştir.
- **Gerekçe:** Simülasyonların amacı şut anını ve oyuncu konumlarını şematik olarak göstermektir. Düşük poligonlu (low-poly) modeller ve sadeleştirilmiş animasyonlar kullanılarak üretim hızı artırılmıştır.

1.1.5 Component Decoupling vs. Synchronization (Bileşen Ayrımı ve Senkronizasyon)

Sistemin özet video, yapay zeka spikeri ve 3D simülasyon gibi farklı modüllerden oluşması, bu çıktıların kullanıcıya sunulurken senkronize edilmesini zorlaştırır.

- **Karar:** Modüller gevşek bağlı (decoupled) tasarlanmış, ancak tüm çıktılar **ortak bir zaman ekseni (t_video_ms)** üzerinden yönetilen bir **Orchestrator** yapısına bağlanmıştır.
- **Gerekçe:** Bu yapı, modüllerin bağımsız geliştirilmesine olanak tanırken, son kullanıcı deneyiminde ses ve görüntünün kaymasını engeller.

1.1.6 Black-Box Automation vs. Explainability (Tam Otomasyon ve Açıklanabilirlik)

VAR (Video Assistant Referee) modülünde sadece "Karar: Faul" çıktısı veren bir model (black-box), kullanıcı için yeterince ikna edici değildir.

- **Karar:** Karar mekanizması **RAG (Retrieval-Augmented Generation)** mimarisi ile desteklenmiştir.
- **Gerekçe:** Sistem sadece karar üretmez; IFAB kural kitabından ilgili maddeleri getirerek kararı "referanslı" hale getirir. Bu, sistemin şeffaflığını ve açıklanabilirliğini artırır.

1.1.7 Completeness vs. Relevance (Kapsayıcılık ve İlgi Çekicilik)

90 dakikalık bir futbol maçı, binlerce pas ve düşük tempolu orta saha mücadelesi içerir. Maçtaki tüm aksiyonları simüle etmek veya özete dahil etmek, son kullanıcı için odak kaybına ve gereksiz veri gürültüsüne (data noise) yol açar.

- **Karar:** Sistem, maçın "tamamını" temsil etmek yerine; sadece **"Yüksek Gol Beklentisi (xG) Olan Atakları"** ve kritik pozisyonları filtreleyerek işler.
- **Gerekçe:** İzleyici ve teknik ekip için asıl değer yaratan anlar, skora etki eden veya gol tehlikesi yaratan aksiyonlardır. Bu filtreleme stratejisi, simülasyon render maliyetlerini %80 oranında düşürürken, özet içeriğinin taktiksel yoğunluğunu ve seyir zevkini artırır.

1.2 Interface documentation guidelines

Bu bölüm, sistemin alt modülleri (Vision, Analytics, VAR, Content Generation) arasındaki veri akışını sağlayan arayüzlerin ve veri yapılarının dokümantasyon standartlarını

tanımlar. Geliştirme sürecinde modüller arası entegrasyonun sorunsuz ilerlemesi için aşağıdaki kurallar zorunlu kılınmıştır.

1.2.1 Data Exchange Format (Veri Değişim Formatı)

- Protokol: Servisler arası iletişim ve veri transferi için JSON (JavaScript Object Notation) formatı standart kabul edilmiştir.
- Blob Veri Yönetimi: Video klipleri, ses dosyaları ve simülasyon çıktıları gibi büyük boyutlu medya verileri, doğrudan JSON payload içine gömülmez. Bu veriler nesne tabanlı depolama (Object Storage / Local File System) üzerinde tutulur; JSON içinde sadece ilgili dosyanın URI/Path referansı taşınır.

1.2.2 Temporal Synchronization Standard (Zaman Senkronizasyonu)

Video işleme, simülasyon ve ses sentezleme modüllerinin çıktılarını tek bir zaman ekseninde birleştirebilmek için ortak bir zaman standardı belirlenmiştir:

- t_video_ms: Tüm zamansal veriler, işlenen maç videosunun başlangıç anını sıfır noktası (0) kabul ederek, geçen süreyi milisaniye (ms) cinsinden tamsayı (integer) olarak ifade eder.
    - *Örnek:* {"event": "goal", "timestamp": 154200} (Maçın 2. dakika 34. saniyesi)

1.2.3 Naming Conventions (İsimlendirme Kuralları)

Kod tabanı ve veri şemalarında tutarlılığı sağlamak için aşağıdaki isimlendirme setleri uygulanır:

- JSON Alanları (Keys): snake_case (Örn: player_id, team_color)
- Sınıf ve Arayüz İsimleri: PascalCase (Örn: VideoProcessor, IVarService)
- Metot ve Fonksiyon İsimleri: Python standartlarına uygun olarak snake_case (Örn: detect_ball, generate_summary)
- API Endpoint Yolları: Küçük harf ve tire kullanımı (kebab-case) (Örn: /api/v1/match-analysis)

1.2.4 API Response Template (Cevap Şablonu)

Tüm modül servisleri, istemciye (Frontend veya diğer servisler) dönerken aşağıdaki standart "Zarf (Envelope)" yapısını kullanır:

JSON

```
{
  "status": "success | error",
  "data": {
    // Başarılı işlem sonucunda üretilen veri yükü (Payload) buraya gelir.
    "match_id": "12345",
    "summary_url": "/storage/clips/summary_12345.mp4"
  },
  "error": {
    // Sadece status="error" durumunda doludur.
    "code": 404,
```

"message": "Video file not found.",
"details": "File ID 12345 does not exist in storage."
}
}

1.2.5 Interface Versioning (Versiyonlama)

Gelecekte yapılacak geliştirmelerin mevcut entegrasyonları bozmaması (Backward Compatibility) için API arayüzleri URI seviyesinde versiyonlanır (Örn: /api/v1/...). Veri şemalarındaki değişikliklerde, mevcut alanları silmek yerine yeni opsiyonel alanlar eklenmesi tercih edilir.

1.3 Engineering standards (e.g., UML and IEEE)

Bu bölüm, sistemin tasarım ve dokümantasyon sürecinde takip edilen mühendislik standartlarını, modelleme dillerini ve kaynak gösterim kurallarını tanımlar. Yazılım geliştirme sürecinde tutarlılığı sağlamak amacıyla UML 2.5 ve IEEE standartlarına sadık kalınmıştır.

1.3.1 Modeling Standards (Modelleme Standartları)

Sistemin yapısal ve davranışsal tasarımını görselleştirmek için aşağıdaki UML diyagram türleri kullanılmıştır:

- UML Class Diagrams: Sistemin statik yapısını, paket hiyerarşisini, sınıflar arası ilişkileri (inheritance, association) ve arayüz tanımlarını göstermek için kullanılır.
- UML Sequence Diagrams: Sistemin iki ana iş akışı olan Highlight Pipeline ve VAR Pipeline süreçlerindeki nesneler arası mesajlaşma trafiğini ve zamanlamayı modellemek için kullanılır.
- UML Activity Diagrams: Karmaşık algoritmik akışların (örn: RAG tabanlı kural eşleştirme veya özet birleştirme mantığı) ve veri işleme adımlarının görselleştirilmesi için gerekli noktalarda kullanılmıştır.

1.3.2 Documentation Format (Sınıf Dokümantasyon Formatı)

Bu raporda yer alan her bir sınıf (Class) tanımı, kodun anlaşılabilirliğini ve uygulanabilirliğini garanti altına almak için aşağıdaki standart şablonu takip eder:

- Responsibility: Sınıfın tekil sorumluluğunu (Single Responsibility Principle) özetleyen kısa açıklama.
- Attributes: Sınıfın sahip olduğu üye değişkenler, veri tipleri ve görünürlük seviyeleri (private/public).
- Public Methods: Sınıfın dış dünyaya açılan metotlarının imzaları, parametre yapıları ve işlev özetleri.
- I/O Data Structures: Metotların kabul ettiği girdi formatları ve ürettiği çıktı veri yapıları.

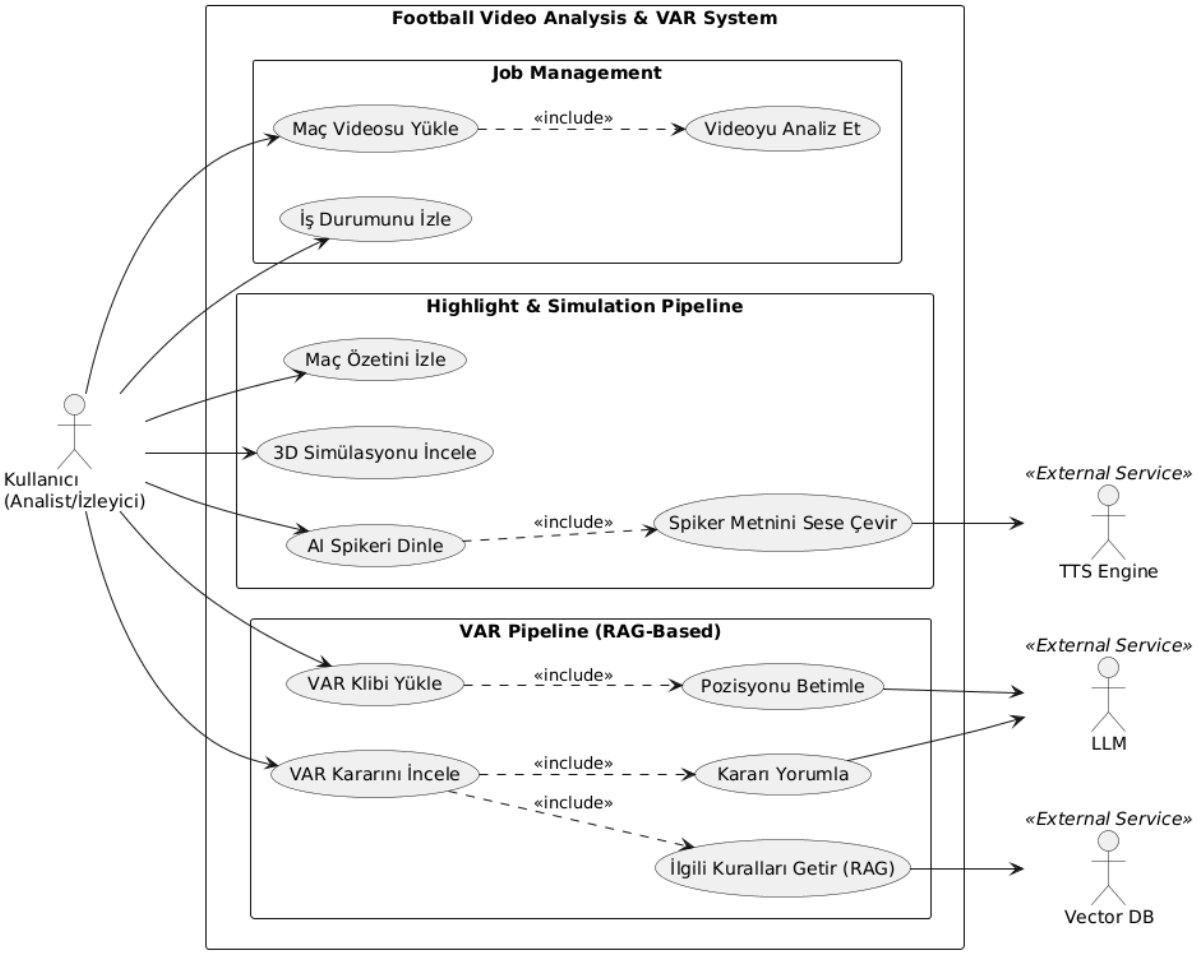
1.3.3 Citation Standards (Referans Standartları)

- Teknik Referanslar: Kullanılan algoritmalar (YOLO, DeepSORT), kütüphaneler ve akademik makaleler IEEE atıf formatına uygun olarak numaralandırılmış ve rapor sonunda listelenmiştir.
- Domain Referansları: Yapay Zeka Hakem modülü için temel alınan kurallar, IFAB Laws of the Game kitapçığına dayandırılır. Bu referanslar, bağlamın gerektirdiği durumlarda ilgili Kural ve Madde numarası belirtilerek detaylandırılır.

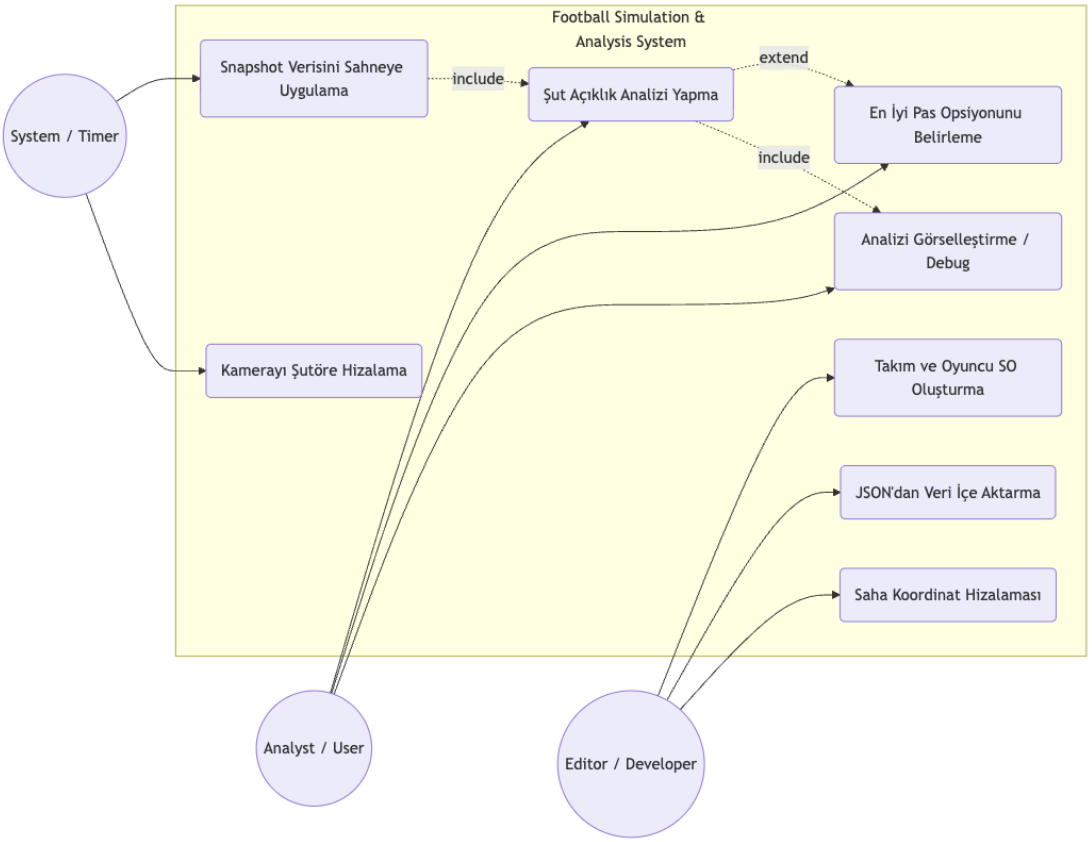
1.4 Definitions, acronyms, and abbreviations

- **VAR:** Video Assistant Referee
- **RAG:** Retrieval-Augmented Generation
- **CV:** Computer Vision
- **Tracking:** Kareler boyunca oyuncu/top konum takibi
- **Highlight:** Maç içindeki önemli anı temsil eden klip
- **Simulation:** Konumsal veriden üretilen görsel temsil
- **AI Spiker:** Özet ve simülasyonla senkron yapay zekâ anlatıcı
- **Homography:** Kamera düzlemi ile saha düzlemi arasındaki dönüşüm
- **Field Calibration:** Saha çizgileri/kesişimlerinden homografi çıkarımı

1.5 Use Cases


Football Video Analysis & VAR System
Job Management
Maç Videosu Yükle
«include»
Videoyu Analiz Et
İş Durumunu İzle
Highlight & Simulation Pipeline
Maç Özetini İzle
3D Simülasyonu İncele
AI Spikeri Dinle
«include»
Spiker Metnini Sese Çevir
VAR Pipeline (RAG-Based)
VAR Klibi Yükle
«include»
Pozisyonu Betimle
VAR Kararını İncele
«include»
Kararı Yorumla
«include»
İlgili Kuralları Getir (RAG)
Kullanıcı
(Analist/İzleyici)
«External Service»
TTS Engine
«External Service»
LLM
«External Service»
Vector DB

Simulation kısmı için use case diagram :



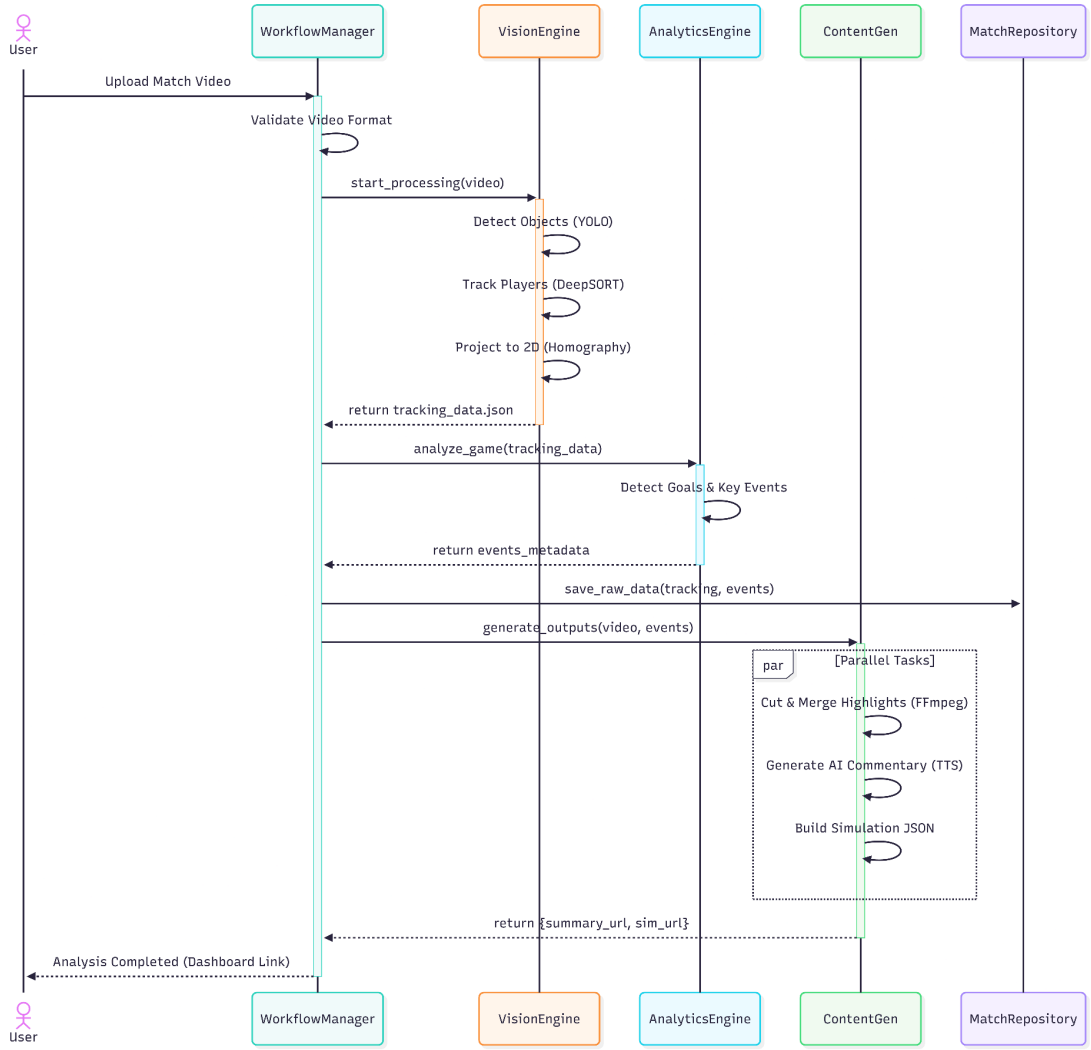
2. Packages

Sistemin mimari tasarımı, **Separation of Concerns** prensibine göre paketlere ayrılmıştır. Bu paketleme yaklaşımı; görüntü işleme, analitik çıkarım, içerik üretimi, VAR karar desteği, veri erişimi ve iş akışı yönetimini birbirinden ayırarak bileşenlerin **gevşek bağlı (loosely coupled)**, bağımsız geliştirilebilir ve test edilebilir olmasını sağlar.

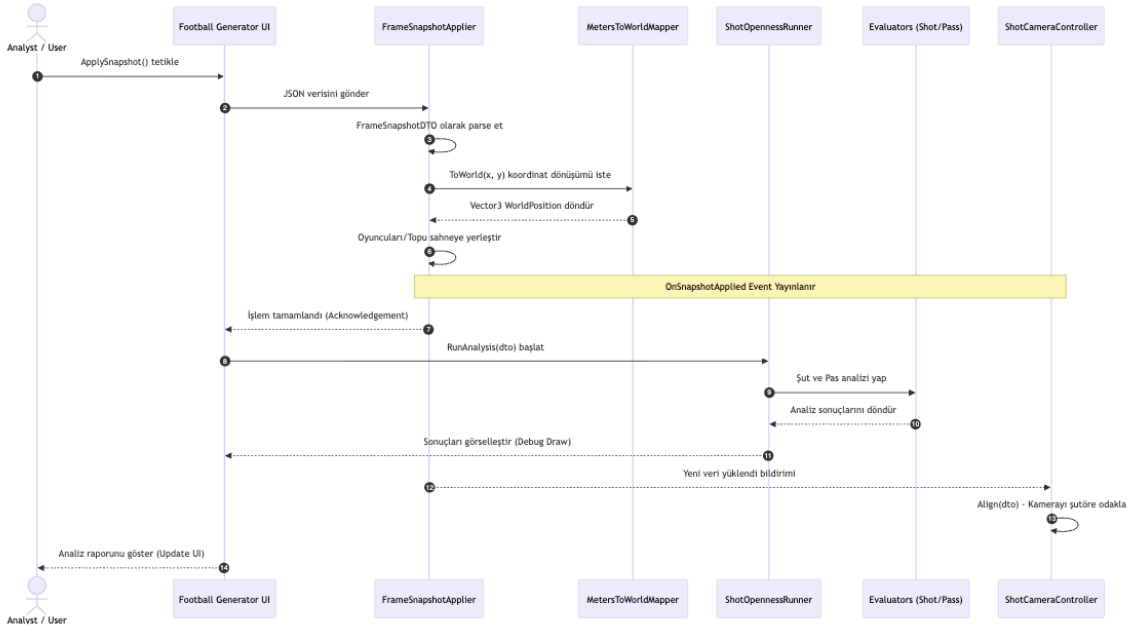
Paketler iki ana katmanda ele alınır:

- **Server**: Video işleme, analiz, simülasyon, üretim ve veri yönetimi yapan çekirdek altyapı

- **Client**: Üretilen çıktıların kullanıcıya senkron şekilde sunulduğu arayüz katmanı


User
WorkflowManager
VisionEngine
AnalyticsEngine
ContentGen
MatchRepository
Upload Match Video
Validate Video Format
start_processing(video)
Detect Objects (YOLO)
Track Players (DeepSORT)
Project to 2D (Homography)
return tracking_data.json
analyze_game(tracking_data)
Detect Goals & Key Events
return events_metadata
save_raw_data(tracking, events)
generate_outputs(video, events)
par
[Parallel Tasks]
Cut & Merge Highlights (FFmpeg)
Generate AI Commentary (TTS)
Build Simulation JSON
return {summary_url, sim_url}
Analysis Completed (Dashboard Link)
User
WorkflowManager
VisionEngine
AnalyticsEngine
ContentGen
MatchRepository

Simulation kısmı için sequence diagram :



2.1 Server

Server katmanı, ham maç videosunu girdi olarak alır ve özet video, simülasyon verileri, AI spiker anlatımı ve VAR açıklamaları gibi çıktıları üretir. Server tarafı altı ana pakete ayrılmıştır.

2.1.1 Video Analysis Package (video_analysis)

Bu paket, ham maç videosunu işleyerek yapılandırılmış sayısal verilere (koordinat, sınıf, takım bilgisi) dönüştüren bilgisayarlı görü bileşenlerini içerir. Sistemin “gözü” olarak işlev görür.

- **ObjectDetector**: YOLO tabanlı tespit modeli için wrapper; oyuncu, top, hakem vb. nesneleri kare bazında tespit eder.

- **TeamTracker**: Tespit edilen oyuncuları forma renk histogramlarına dayalı K-Means clustering ile HOME/AWAY olarak ayırır ve DeepSORT ile kareler arası takip ederek konumsal zaman serisi üretir.

- **FieldCalibrator**: Saha çizgilerini tespit ederek homography matrisini hesaplar; görüntü koordinatlarını (u, v) saha koordinatlarına (x, y) dönüştürür (opsiyonel/fallback destekli).

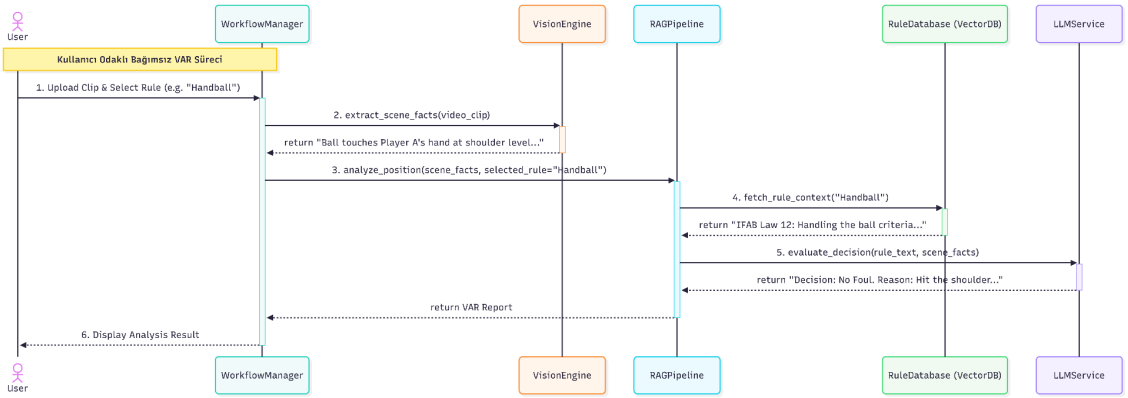
2.1.2 Analytics Package (analytics_engine)

Bu paket, Vision katmanından gelen ham konumsal verileri ve sinyalleri işleyerek olay/taktik düzeyinde anlam çıkaran mantık katmanıdır.

- **EventDetector**: Topun hız/yön/konum parametrelerinden “Şut”, “Gol” gibi olayları rule-based heuristics ile tespit eder.

- **PossessionCalculator**: Topun hangi takımda kaldığını kare bazlı hesaplayarak topla oynama yüzdelerini çıkarır.

2.1.3 VAR Package (var_system)



Bu paket, özet akışından bağımsız çalışan, tartışmalı pozisyonları kural kitabına göre değerlendiren RAG tabanlı karar destek modülüdür.

- **RAGPipeline**: Pozisyon tanımı/özeti ile IFAB kural veritabanı arasında semantik arama akışını yürütür.

- **RuleRetriever**: Vector DB’den en alakalı kural maddelerini getirir (örn. “Law 12 – Fouls and Misconduct”).

- **DecisionExplainer**: LLM ile etkileşime girerek, getirilen kurallar ve pozisyon bağlamı ile açıklanabilir VAR raporu üretir.

2.1.4 Content Generation Package (content_gen)

Bu paket, analiz sonuçlarını son kullanıcının tüketeceği görsel/işitsel formatlara dönüştürür.

- **VideoSummarizer**: Önemli an timestamp'lerine göre ham videodan klipleri keser ve birleştirir (ffmpeg wrapper).

- **NarratorService (AI Spiker)**: Olay akışına göre dinamik anlatım metni oluşturur; opsiyonel olarak TTS ile ses dosyası üretir.

- **SimDataBuilder**: Tracking verisini Unity simülasyon motorunun kabul edeceği optimize JSON formatına dönüştürür.

2.1.5 Data Access Package (data_layer)

Bu paket, sistemin veritabanı ve dosya/nesne depolama ile etkileşimini yönetir.

- **MatchRepository**: Maç metadata ve analiz sonuçlarını ilişkisel veritabanına (PostgreSQL) yazar/okur.

- **StorageManager**: Özet video, klipler, simülasyon çıktıları, ses dosyaları gibi büyük medya varlıklarının dosya sistemi veya bulut depolamaya kaydını yönetir.

2.1.6 Simulation Package (simulation_engine)

Simulation paketi, analiz katmanından veya JSON tracking çıktılarından gelen konumsal verileri Unity ortamında yeniden yapılandırarak şut, pas ve pozisyon değerlendirme senaryolarını görselleştiren ve metrik üreten modüldür.

Bu paket, görsel debug, geometri tabanlı açıklık hesaplama (shot openness), pas alternatif analizi ve kamera hizalama işlevlerini içerir.

- **ShotOpennessEvaluator**:

  Şut atan oyuncunun kaleye olan görüş açısını hesaplar. Kale direkleri arasındaki açıyı belirler, rakip oyuncular ve kalecinin oluşturduğu engel dairelerini projekte eder ve açık kalan açısal segmentleri hesaplayarak "açıklık oranı" üretir.

- **ShotEvaluator2D**:

  Hesaplanan açıklık oranı ve mesafe/açı parametrelerini kullanarak basit olasılık modeli üzerinden gol ihtimali skorunu üretir.

- **BestPassEvaluator**:

  Şut yerine pas alternatiflerini değerlendirir. Takım arkadaşları için ayrı ayrı şut

açıklığı simülasyonu çalıştırır ve en avantajlı pas seçeneğini belirler.

- **PassEvaluator2D**:

  Pas hattı üzerinde rakip engel kontrolü yapar. Shooter ile takım arkadaşı arasındaki doğrusal hattın bloklanıp bloklanmadığını belirler.

- **ShotOpennessRunner**:

  Simülasyon akışını yöneten sınıftır. FrameSnapshotDTO'yu alır, gerekli evaluator sınıflarını çalıştırır ve sonuçları üst katmanlara iletir.

- **ShotDebugDrawer**:

  Simülasyonun görselleştirme katmanıdır. Shooter'dan kaleye çizgiler çizer, engel dairelerini gösterir ve açık alan wedge mesh'lerini üretir.

2.2 Client

Client katmanı, server tarafından üretilen tüm çıktıları (özet video, simülasyon, AI spiker anlatımı, VAR raporu) kullanıcıya sunar. Client tarafında temel amaç, farklı çıktıları **tek bir zaman çizelgesi (t_video_ms)** üzerinde senkron göstermektir.

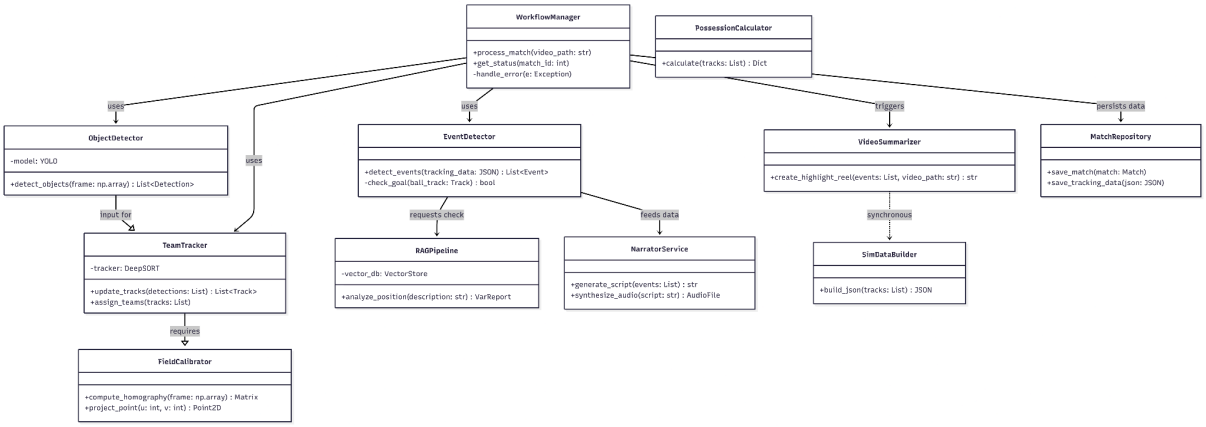
2.2.1 View Package (ui)

- **UploadView**: Maç videosu yükleme ve job başlatma.
- **SummaryPlayerView**: Özet video oynatımı + highlight listesi.
- **SimulationView**: Şut simülasyonu oynatımı (özet ile ilişkili).
- **NarrationView**: Anlatım metni/ses segmentlerinin senkron gösterimi.
- **VARAnalysisView**: VAR raporu + kural referansları + kanıtların gösterimi.
- **JobStatusView**: Pipeline ilerleme durumu ve çıktılara erişim.

2.2.2 Sync Package (sync/)

- **SyncCoordinator**: Video–simülasyon–anlatım bileşenlerini t_video_ms üzerinden hizalar.

- **MasterClock**: Oynatıcı zamanını tek kaynak kabul ederek diğer bileşenleri sürer.



3. Class Interfaces

3.1 Server

3.1.1 Video Analysis Package (video_analysis)

3.1.1.1 ObjectDetector

class ObjectDetector
ultralytics kütüphanesini sarmalayarak (wrapper) YOLOv8 modelini yönetir. Video kareleri üzerindeki nesne tespitini (Inference) gerçekleştirir, GPU kaynaklarını yönetir ve düşük güven skoruna sahip hatalı tespitleri filtreler.
Attributes
private ultralytics.YOLO model
private str device

private float conf_threshold
private Dict[int, str] class_names
Methods
__init__(model_path: str, device: str)
process_frame(frame: np.ndarray) -> List[Detection]
process_batch(frames: List) -> List[List[Detection]]
release()

3.1.1.2 TeamTracker

class TeamTracker
Nesne tespiti sonuçlarını zamansal olarak ilişkilendirerek (association) her oyuncuya benzersiz bir ID atar (DeepSORT). Ayrıca oyuncu görüntülerinden renk analizi yaparak (K-Means Clustering) oyuncuları Takım A ve Takım B olarak sınıflandırır.
Attributes
private DeepSORT tracker
private Dict[int, KMeans] team_clusters
private Dict[int, List] track_history
Methods
update_tracks(detections: List, frame: np.ndarray)
assign_teams(tracks: List, frame: np.ndarray)
get_active_tracks() -> List[Track]

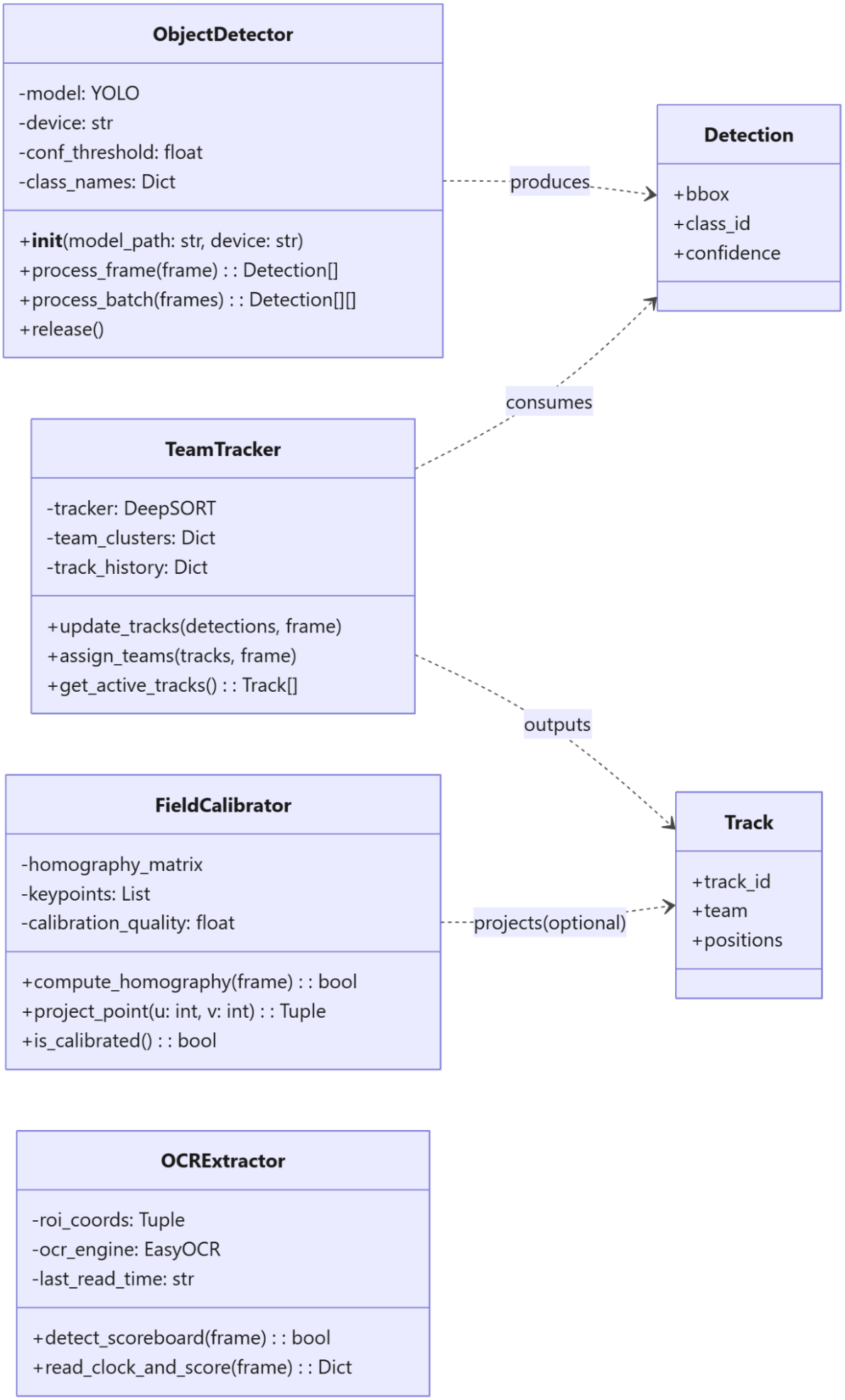
3.1.1.3 FieldCalibrator

class FieldCalibrator
Kamera görüntüsündeki perspektif bozulmasını (trapezoid) düzelterek, oyuncu ve top konumlarını metrik 2D saha koordinatlarına (kuş bakışı) dönüştürür.
Attributes

private np.ndarray homography_matrix
private List[Point] keypoints
private float calibration_quality
Methods
compute_homography(frame: np.ndarray) -> bool
project_point(u: int, v: int) -> Tuple[float, float]
is_calibrated() -> bool

3.1.1.4 OCRExtractor

class OCRExtractor
Video üzerindeki skor tabelası (scoreboard) bölgesini izleyerek maç skoru ve dakika bilgisini Optik Karakter Tanıma (OCR) yöntemiyle metin formatına çevirir.
Attributes
private Tuple roi_coords
private EasyOCR ocr_engine
private str last_read_time
Methods
detect_scoreboard(frame: np.ndarray) -> bool
read_clock_and_score(frame: np.ndarray) -> Dict


ObjectDetector
-model: YOLO
-device: str
-conf_threshold: float
-class_names: Dict
+init(model_path: str, device: str)
+process_frame(frame) : : Detection[]
+process_batch(frames) : : Detection[][]
+release()
produces
Detection
+bbox
+class_id
+confidence
consumes
TeamTracker
-tracker: DeepSORT
-team_clusters: Dict
-track_history: Dict
+update_tracks(detections, frame)
+assign_teams(tracks, frame)
+get_active_tracks() : : Track[]
outputs
FieldCalibrator
-homography_matrix
-keypoints: List
-calibration_quality: float
+compute_homography(frame) : : bool
+project_point(u: int, v: int) : : Tuple
+is_calibrated() : : bool
projects(optional)
Track
+track_id
+team
+positions
OCRExtractor
-roi_coords: Tuple
-ocr_engine: EasyOCR
-last_read_time: str
+detect_scoreboard(frame) : : bool
+read_clock_and_score(frame) : : Dict

3.1.2 Tracking Package (tracking)

class BotSortTrackerAdapter
Oyuncular, hakemler ve kaleciler için BoT-SORT algoritmasını kullanarak çoklu nesne takibi (Multi-Object Tracking) gerçekleştirir; kare bazlı tespitleri sürekli izlere dönüştürür.
Attributes
float track_threshold
float match_threshold
int track_buffer
Methods
+ track(detections_by_frame : dict) : list[TrackRecord] — Kare bazlı tespitleri girdi alarak, top dışındaki tüm nesne sınıfları için track_id içeren iz kayıtları üretir

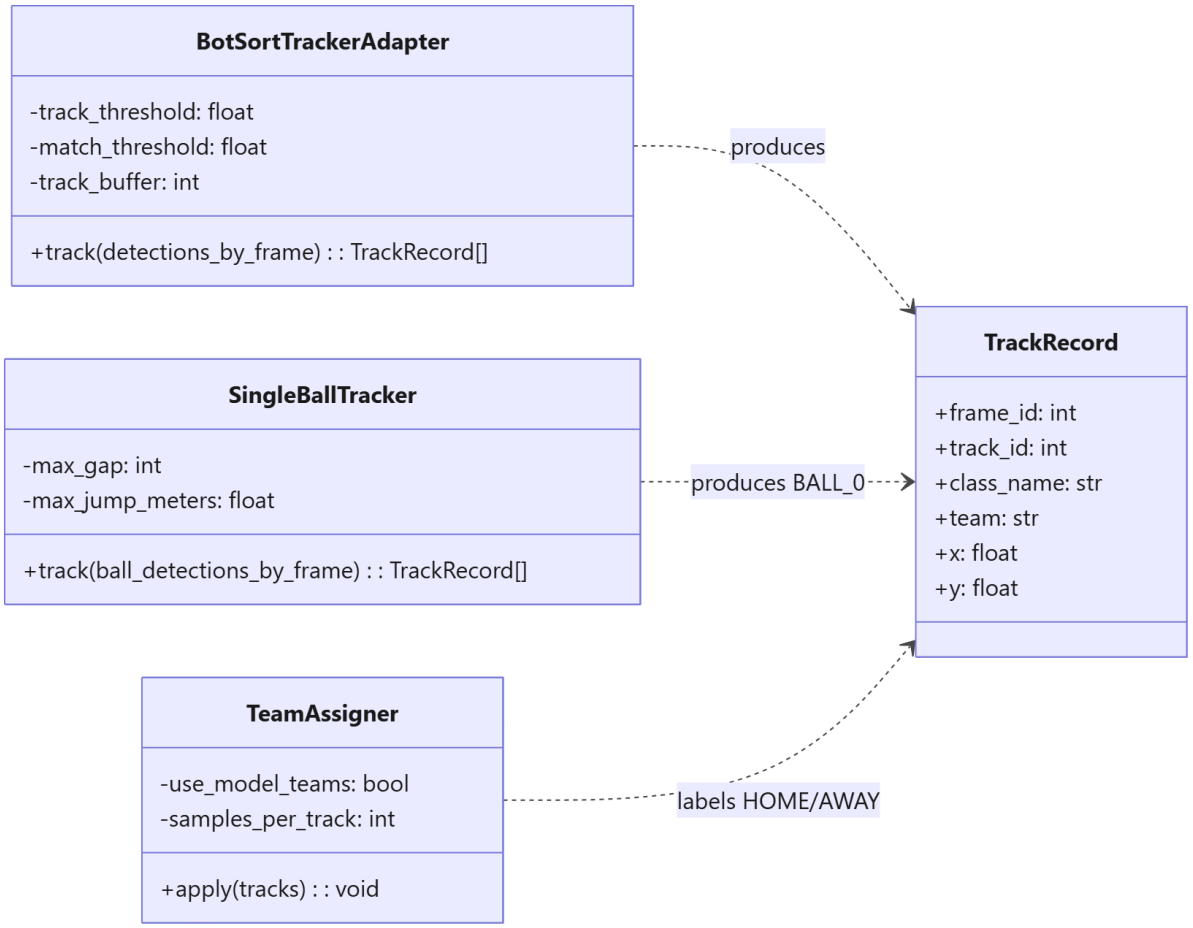
---

class SingleBallTracker
Sahadaki çoklu top adayları arasından en olası olanı seçer ve kısa süreli veri kesintilerinde sabit hız (constant-velocity) tahminiyle yörüngeyi tamamlayan özelleştirilmiş top takibi yapar.
Attributes
int max_gap
float max_jump_meters
Methods
+ track(ball_detections_by_frame : dict) : list[TrackRecord] — Top adaylarını süzerek sistem için tekil bir top izi (BALL_0) oluşturur

---

class TeamAssigner
Takip edilen oyuncu izlerine HOME veya AWAY takım etiketlerini atar; bunu model çıktıları veya forma renk kümelemesi (clustering) üzerinden gerçekleştirir.

Attributes
bool use_model_teams
int samples_per_track
Methods
+ apply(tracks : list) : void — Girdi olarak aldığı iz listesini güncelleyerek her kayıt için team alanını doldurur



3.1.3 VAR Package (var_system)

3.1.3.1 VARClipIngestor

class VARClipIngestor
Video veya görüntü dosyasını alarak doğrulamasını yapar ve LLM API üzerinden pozisyonun metin açıklamasını (description) çıkarır. Dosya boyutu (maks. 20 MB) ve MIME tipi kontrolü yapıldıktan sonra AI modeline gönderilir. Model, pozisyonun türü, oyuncular, temas bilgisi, topun konumu ve şiddeti gibi bilgileri Türkçe metin olarak döndürür. Safety settings kontrolleri de bu sınıf tarafından yönetilir.
Attributes
private bytes video_bytes
private str mime_type
private str filename
private float file_size_mb
private float MAX_FILE_SIZE_MB
private list<str> SUPPORTED_MIME_TYPES
private GenerativeModel ai_model
private list<dict> safety_settings
Methods
+ validate() : bool — Dosya boyutu ve MIME tipi doğrulaması yapar.
+ ingest() : str — Ana giriş noktası: doğrula → çıkar → açıklama döndür.
+ extract_description() : str — LLM API'ye video gönderip metin açıklama alır.
+ get_video_part() : dict — Base64 encode edilmiş inline_data dict'i döndürür.
+ get_file_info() : dict — Dosya bilgilerini (ad, boyut, tip) döndürür.

- _check_file_size() : bool — Boyut kontrolü (≤ 20 MB).
- _check_mime_type() : bool — MIME tipi kontrolü.
- _encode_base64() : str — video_bytes → Base64 string dönüşümü.
- _build_extraction_prompt() : str — Pozisyon analiz prompt'u oluşturur.
- _parse_response(response) : str — LLM yanıtını parse eder, safety kontrol yapar.

3.1.3.2 RuleRetriever

class RuleRetriever
Vektör veritabanı üzerinden IFAB futbol kurallarını arar ve döndürür. İlk başlatmada PDF dosyasından (oyun-kurallari.pdf) sayfa bazlı chunk'lar çıkarılarak vektör veritabanına indexlenir. PDF bulunamazsa hardcoded IFAB kuralları kullanılır. Arama sırasında sorgu metni vektör benzerliğiyle eşleşen en ilgili kurallar döndürülür. Manuel kural seçimi yapıldığında frontend seçimini kural ID'sine eşler (örn: "law12" → "law12_fauller").
Attributes
private PersistentClient vector_db_client
private Collection rules_collection
private list<dict> IFAB_RULES
private str pdf_path
private bool is_initialized
private int collection_count
Methods
+ init_rag() : dict — RAG sistemini başlatır (PDF → vektör veritabanı veya hardcoded fallback).
+ search(query: str, n_results: int) : SearchResult — Sorguya en uygun kuralları arar.
+ get_rule_by_id(rule_id: str) : str — Belirli kural ID'sine göre kural içeriğini

döndürür.
+ get_rule_by_selection(selection: str) : tuple — Frontend seçimini kural ID + talimat çiftine eşler.
+ get_status() : dict — RAG durumunu (count, rules listesi) döndürür.
+ get_collection_count() : int — Koleksiyondaki döküman sayısını döndürür.
- _load_from_pdf() : list<dict> — PDF'ten sayfa bazlı chunk'lar çıkarır.
- _load_hardcoded_rules() : None — Hardcoded kuralları vektör veritabanına ekler.
- _extract_text_from_pdf(pdf_path: str) : list<dict> — PDF metin çıkarma işlemi.
- _chunk_pages(reader: PdfReader) : list<dict> — Sayfa bazlı chunking.
- _upsert_to_collection(chunks: list) : None — Chunk'ları vektör veritabanına yazar.

3.1.3.3 DecisionExplainer

class DecisionExplainer
Video, metin açıklama ve ilgili IFAB kurallarını birlikte değerlendirerek nihai VAR kararını üretir. LLM API'ye profesyonel FIFA sertifikalı VAR hakemi rolüyle prompt gönderilir. Manuel kural seçimi aktifse sadece seçilen kural kapsamında değerlendirme yapılır. Yanıt JSON formatında parse edilerek TechnicalAnalysis, RuleInterpretation, VARAssessment ve Decision alt modellerine ayrılır. Kart kararları IFAB Law 12'ye göre (Dikkatsiz → Faul, Tedbirsiz → Sarı, Aşırı Güç → Kırmızı) belirlenir.
Attributes
private GenerativeModel ai_model
private list<dict> safety_settings
private str model_name
Methods
+ make_decision(video_bytes, mime_type, description, rag_rules, manual_selection) : AnalysisResult — Ana karar verme fonksiyonu.

+ build_prompt(description, rag_rules, manual_selection) : str — Analiz prompt'unu oluşturur.
+ parse_decision(response_text: str) : AnalysisResult — JSON yanıtı parse eder.
+ get_confidence(result: AnalysisResult) : int — Güven skorunu döndürür (0-100).
- _build_strict_instruction(manual_selection: str) : str — Manuel seçim kısıtlama talimatı.
- _build_card_rules_prompt() : str — Kart kuralları prompt bloğu.
- _build_json_schema_prompt() : str — JSON çıktı şema tanımı.
- _extract_json(text: str) : dict — Yanıttan JSON bloğu çıkarır.
- _validate_result(parsed: dict) : bool — Zorunlu alanları doğrular.
- _create_fallback_result(error: str) : AnalysisResult — Hata durumunda varsayılan sonuç.

class TechnicalAnalysis
Pozisyondaki fiziksel temasın teknik detaylarını tutar.
Attributes
private bool hasContact
private str contactType
private str severity
private str location
Methods
Getter and setter methods.

class RuleInterpretation
Uygulanan IFAB kuralının yorumunu tutar.
Attributes

private str rule
private str ruleName
private str explanation
Methods
Getter and setter methods.

class VARAssessment
VAR müdahale değerlendirmesini tutar.
Attributes
private bool interventionNeeded
private str reason
Methods
Getter and setter methods.

class Decision
Nihai VAR kararını tutar.
Attributes
private str type
private str main
private str sub
private str icon
Methods
Getter and setter methods.

class AnalysisResult

DecisionExplainer'ın ürettiği tüm analiz sonuçlarını bir arada tutar.
Attributes
private str summary
private TechnicalAnalysis technicalAnalysis
private RuleInterpretation ruleInterpretation
private VARAssessment varAssessment
private Decision decision
private int confidence
private str notes
Methods
Getter and setter methods.

3.1.3.4 EvidencePackager

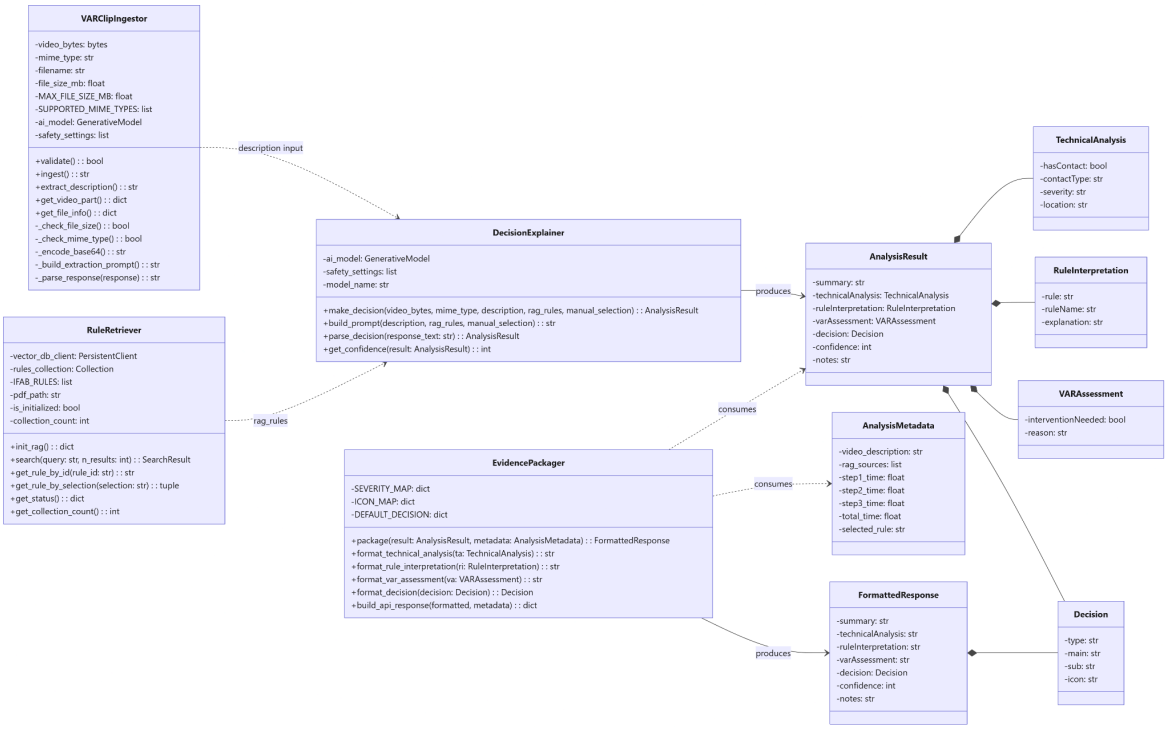
class EvidencePackager
DecisionExplainer'dan gelen ham analiz sonuçlarını frontend'e uygun HTML formatına dönüştürür ve API yanıtını paketler. TechnicalAnalysis HTML <ul> listesine, RuleInterpretation ve VARAssessment HTML paragraflarına çevrilir. Decision nesnesine uygun ikon eklenir. Severity kodları Türkçe metne çevrilir (light → "Düşük - Normal oyun teması", severe → "Yüksek - Tehlikeli müdahale"). Eksik alanlar varsayılan değerlerle doldurulur.
Attributes
private dict SEVERITY_MAP
private dict ICON_MAP
private dict DEFAULT_DECISION
Methods
+ package(result: AnalysisResult, metadata: AnalysisMetadata) : FormattedResponse — Ana paketleme fonksiyonu.

+ format_technical_analysis(ta: TechnicalAnalysis) : str — TechnicalAnalysis → HTML listesi.
+ format_rule_interpretation(ri: RuleInterpretation) : str — RuleInterpretation → HTML paragrafları.
+ format_var_assessment(va: VARAssessment) : str — VARAssessment → HTML paragrafları.
+ format_decision(decision: Decision) : Decision — Decision'a ikon ekler.
+ build_api_response(formatted, metadata) : dict — Frontend'e gönderilecek tam API yanıtı.
- _severity_to_text(severity: str) : str — Severity kodunu Türkçe metne çevirir.
- _decision_to_icon(decision_type: str) : str — Karar tipini ikona eşler.
- _ensure_defaults(result: dict) : dict — Eksik alanları varsayılanlarla doldurur.

class AnalysisMetadata
Analiz süreç bilgilerini (zamanlama, kaynaklar) tutar.
Attributes
private str video_description
private list<str> rag_sources
private float step1_time
private float step2_time
private float step3_time
private float total_time
private str selected_rule
Methods
Getter and setter methods.

class FormattedResponse

Frontend'e gönderilecek formatlanmış yanıtı tutar.
Attributes
private str summary
private str technicalAnalysis
private str ruleInterpretation
private str varAssessment
private Decision decision
private int confidence
private str notes
Methods
Getter and setter methods.



3.1.4 Content Generation Package

3.1.4.1 VideoSummarizer

class VideoSummarizer

Tespit edilen önemli olayların zaman damgalarını kullanarak ham videodan ilgili klipleri keser ve ffmpeg wrapper kullanarak tek bir özet video dosyası haline getirir.
Attributes
private Tuple[int, int] output_resolution
private float min_clip_duration
private str output_format
Methods
create_highlight_reel(video_path: str, events: List[Event]) -> str
_cut_segment(video_path: str, start: int, end: int) -> str
_merge_segments(segment_paths: List[str]) -> str

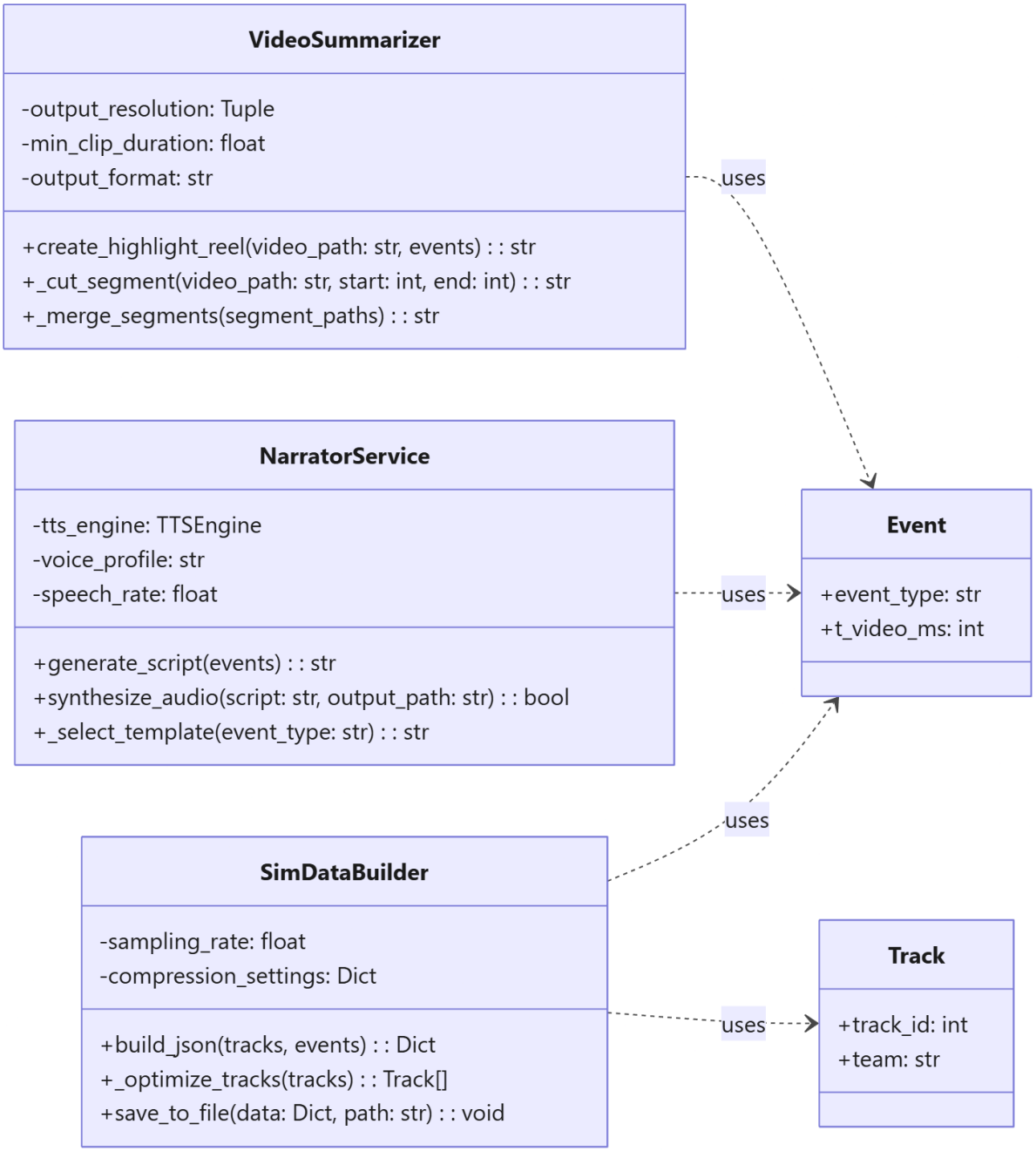
3.1.4.2 NarratorService

class NarratorService
Maçın olay akışına (gol, şut, faul) göre dinamik ve doğal dilde bir anlatım metni (script) oluşturur. İstenirse bu metni TTS (Text-to-Speech) motoru ile ses dosyasına dönüştürür.
Attributes
private TTSEngine tts_engine
private str voice_profile
private float speech_rate
Methods
generate_script(events: List[Event]) -> str
synthesize_audio(script: str, output_path: str) -> bool
_select_template(event_type: str) -> str

3.1.4.3 SimDataBuilder

class SimDataBuilder

Takip (tracking) ve olay (event) verilerini işleyerek, Unity tabanlı simülasyon motorunun okuyabileceği optimize edilmiş JSON formatına dönüştürür. Gereksiz kareleri (downsampling) eler.
Attributes
private float sampling_rate
private Dict compression_settings
Methods
build_json(tracks: List[Track], events: List[Event]) -> Dict
_optimize_tracks(tracks: List[Track]) -> List[Track]
save_to_file(data: Dict, path: str) -> void



3.1.5 Data Access Package (data_layer)

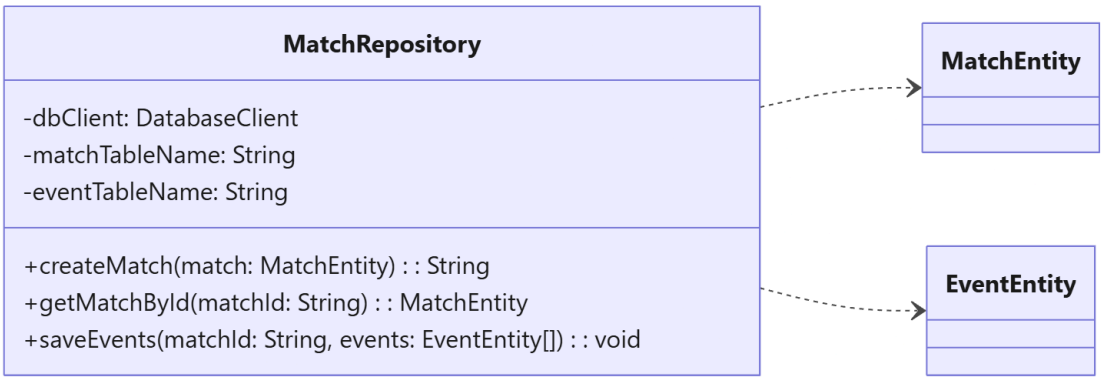
3.1.5.1 MatchRepository

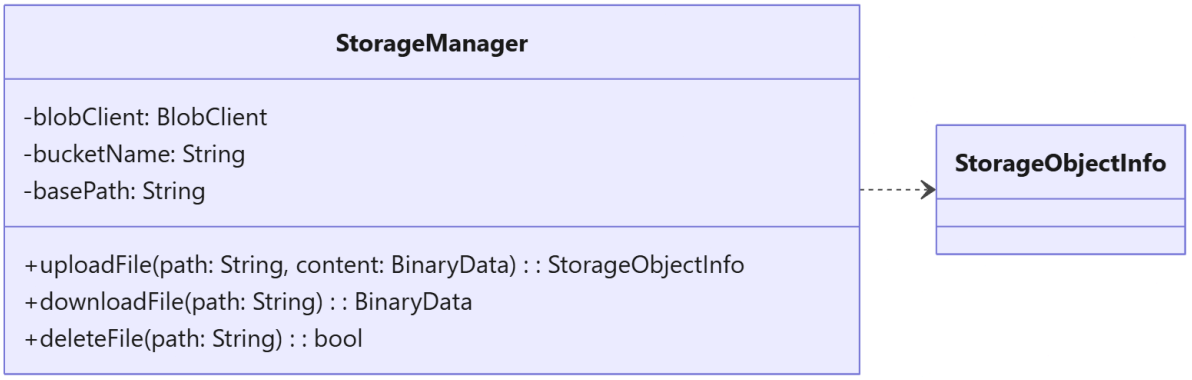
class MatchRepository
Maç, olay, analiz çıktısı ve job durum verilerinin kalıcı veri katmanında yönetimini sağlayan sınıf.

Attributes
private dbClient: DatabaseClient
private matchTableName: String
private eventTableName: String
Methods
+ createMatch(match: MatchEntity) : String
+ getMatchById(matchId: String) : MatchEntity
+ saveEvents(matchId: String, events: EventEntity[]) : void

3.1.5.2 StorageManager

class StorageManager
Video, özet klip, simülasyon çıktısı ve rapor dosyalarının obje depolamada yönetimini sağlayan sınıf.
Attributes
private blobClient: BlobClient
private bucketName: String
private basePath: String
Methods
+ uploadFile(path: String, content: BinaryData) : StorageObjectInfo
+ downloadFile(path: String) : BinaryData
+ deleteFile(path: String) : bool





3.1.6 Simulation Package (simulation_engine)

3.1.6.1 TeamDefinitionSO

class TeamDefinitionSO
Takımların kimlik bilgilerini ve görsel kimliklerini (renk, logo) yöneten veri modelidir.
Attributes
public str teamId — "HOME", "AWAY" gibi benzersiz kimlik.
public str teamName — Takımın tam adı.

public str shortCode — Skor tabelası için kısa kod.
public Color primaryColor — Oyuncu gövde rengi.
public Color secondaryColor — Oyuncu baş rengi.
public Sprite logo — Takım amblemi.

3.1.6.2 PlayerDefinitionSO

class PlayerDefinitionSO
Tekil bir oyuncunun statik verilerini, takım bağlılığını ve görsel prefab referansını tutar.
Attributes
public str playerId — Veri setinden gelen benzersiz oyuncu ID'si.
public str playerName — Oyuncu adı.
public int jerseyNumber — Forma numarası (0-99).
public TeamDefinitionSO team — Oyuncunun bağlı olduğu takım.
public PlayerRole role — GK, DEF, MID, FWD rollerinden biri.

public float interceptRadiusMeters — Müdahale/Blok yarıçapı.
public GameObject visualPrefab — Sahada oluşturulacak 3D model.

3.1.6.3 MetersToWorldMapper

class MetersToWorldMapper
Tracking verisindeki metre cinsinden koordinatları Unity dünya (world) koordinatlarına dönüştürür. Eksen eşleşmelerini ve saha merkezleme işlemlerini yönetir.
Attributes
public float fieldWidth — Saha eni (68m).
public float fieldLength — Saha boyu (105m).
public bool originAtCenter — Merkezin (0,0,0) olup olmayacağı.
public bool lengthAlongX — Saha boyunun X ekseninde olup olmadığı.
Methods
+ ToWorld(float x, float y) : Vector3 — Metre verisini Unity vektörüne çevirir.

3.1.6.4 FrameSnapshotApplier

class FrameSnapshotApplier
JSON verisinden gelen anlık durumu sahneye uygular. Oyuncuları yerleştirir, şut atan oyuncuyu (shooter) işaretler ve topun konumunu günceller.
Attributes
public TextAsset frameJson — Analiz edilecek JSON dosyası.
public MetersToWorldMapper mapper — Koordinat dönüştürücü referansı.
public Transform runtimeRoot — Sahne objelerinin ebeveyn objesi.
private Dictionary<int, GameObject> _players — Mevcut oyuncu havuzu.
Methods
+ ApplySnapshot() — JSON'u parse eder ve sahneye nesneleri dizer.
- GetOrCreate(int id) : GameObject — Oyuncu objesini havuzdan alır veya oluşturur.
- EnsureShooterMarker(Transform player) — Shooter üzerine görsel marker ekler.

3.1.6.5 ShotCameraController

class ShotCameraController
Şut analizini kolaylaştırmak için kamerayı shooter ve kale arasındaki hatta dinamik olarak konumlandırır.
Attributes
public Camera cam — Kullanılacak ana kamera.
public float backOffset — Shooter'dan ne kadar geride duracağı.
public float height — Kamera yüksekliği.
Methods
+ Align(FrameSnapshotDTO dto) — Kamerayı şut açısına göre hizalar.

3.1.6.6 FootballSOGeneratorWindow

class FootballSOGeneratorWindow
Editör içinde takımları, oyuncuları ve prefabları manuel veya JSON üzerinden toplu oluşturmaya yarayan araç penceresidir.

Methods
+ Open() — Editör penceresini açar.
+ GenerateAllManual() — Girilen parametrelere göre ScriptableObject'leri oluşturur.
+ ImportFromJson() — JSON dosyasından tüm takım ve oyuncu yapısını içe aktarır.
- CreateOrUpdateSnowmanPrefab(string path) : GameObject — Temel 3D oyuncu modelini oluşturur.

3.1.6.7 ShotGeometryConfig

class ShotGeometryConfig
Saha ölçülerini (FIFA standartları) ve şut analizi için gerekli olan oyuncu/kaleci blok yarıçaplarını merkezi olarak tutan yapılandırma sınıfıdır.
Attributes
public float fieldWidth — Saha eni (68m).
public float fieldLength — Saha boyu (105m).
public float goalWidth — Kale genişliği (7.32m).

public float playerBlockRadius — Oyuncu müdahale mesafesi.
public float gkBlockRadius — Kaleci müdahale mesafesi.

3.1.6.8 ShotOpennessEvaluator

class ShotOpennessEvaluator
Şutörün kaledeki açık açısını hesaplayan temel algoritma sınıfıdır. Engellerin (rakip ve kaleci) kapattığı açısal aralıkları (blocked intervals) belirleyerek toplam şut açıklığı oranını üretir.
Methods
+ EvaluateOpenness(dto: FrameSnapshotDTO) : float — Şut açıklığı oranını hesaplar.
- MergeIntervals(intervals: List) — Üst üste binen kapalı açıları birleştirir.

3.1.6.9 BestPassEvaluator

class BestPassEvaluator
Şut çekmek yerine takım arkadaşlarına pas vermenin daha avantajlı olup olmadığını analiz eder. Her pas opsiyonu için pas yolunun açıklığını ve pas sonrası şut ihtimalini değerlendirir.

Methods
+ FindBestPass(dto: FrameSnapshotDTO) : PassResult — En iyi pas adayını ve skorunu döndürür.

3.1.6.10 ShotOpennessRunner

class ShotOpennessRunner
Tüm analiz sürecini (şut ve pas değerlendirme) yöneten ve sonuçları diğer modüllere (UI/Debug) aktaran ana kontrolör sınıfıdır.
Methods
+ RunAnalysis(dto: FrameSnapshotDTO) — Tüm evaluatörleri sırayla çalıştırır.

3.1.6.11 PitchAutoAlign

class PitchAutoAlign
Sahadaki 3D plane objesini `MetersToWorldMapper` verilerine göre otomatik olarak ölçeklendirir ve konumlandırır.
Attributes
public Transform pitchPlane — Hizalanacak saha objesi.

public float unityPlaneSize — Plane mesh'in temel boyutu (10x10).
Methods
+ AlignNow() — Sahayı mapper verilerine göre boyutlandırır.

3.1.6.12 PlayerVisual

class PlayerVisual
Oyuncunun 3D modelinin (Snowman) görsel özelliklerini (takım renkleri vb.) yöneten sınıftır.
Methods
+ ApplyDefinition(def: PlayerDefinitionSO) — Takım renklerini materyallere uygular.
+ SetRenderers(body, head) — Gövde ve baş render referanslarını atar.

3.1.6.13 ShotDebugDrawer

class ShotDebugDrawer
Hesaplanan şut açılarını, engel dairelerini ve açık alan "wedge" (yeşil/kırmızı alanlar) geometrilerini Gizmos veya Mesh kullanarak sahada görselleştirir.

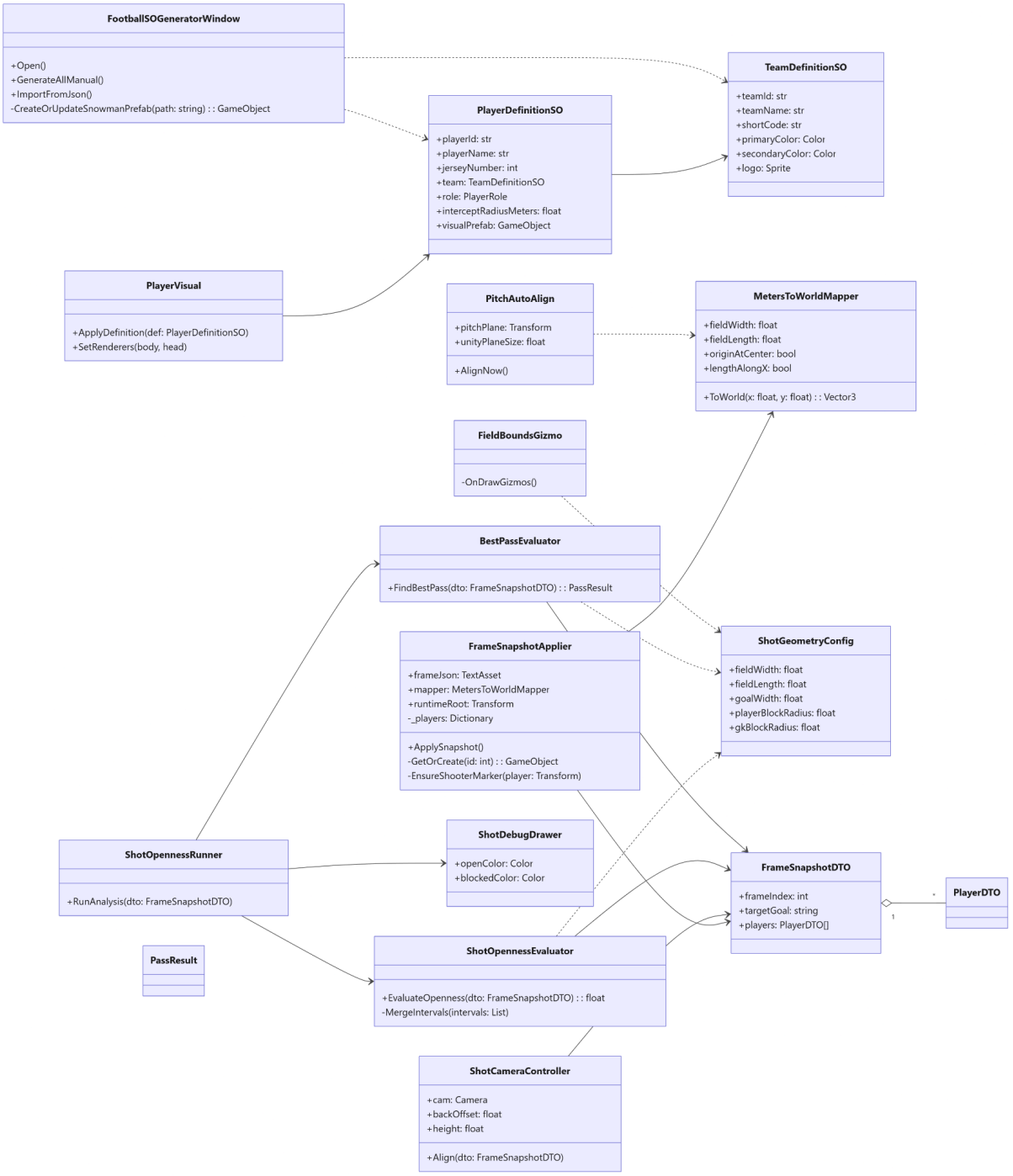
Attributes
public Color openColor — Açık alan rengi (Yeşil).
public Color blockedColor — Kapalı alan rengi (Kırmızı).

3.1.6.14 FrameSnapshotDTO

class FrameSnapshotDTO
Tracking verisinden gelen JSON verisini tutan veri transfer nesnesidir. İçerisinde Shooter, Goalkeeper, Ball ve Players sınıflarını barındırır.
Attributes
public int frameIndex — Verinin kare numarası.
public string targetGoal — Hedef kale ("TOP"/"BOTTOM").
public PlayerDTO[] players — Sahadaki tüm oyuncuların listesi.

3.1.6.15 FieldBoundsGizmo

class FieldBoundsGizmo
Editör ekranında sahanın sınırlarını sarı çizgilerle çizerek görsel bir referans sağlar.
Methods
- OnDrawGizmos() — Saha sınırlarını sahne ekranında çizer.


FootballSOGeneratorWindow
+Open()
+GenerateAllManual()
+ImportFromJson()
-CreateOrUpdateSnowmanPrefab(path: string) : : GameObject
TeamDefinitionSO
+teamId: str
+teamName: str
+shortCode: str
+primaryColor: Color
+secondaryColor: Color
+logo: Sprite
PlayerDefinitionSO
+playerId: str
+playerName: str
+jerseyNumber: int
+team: TeamDefinitionSO
+role: PlayerRole
+interceptRadiusMeters: float
+visualPrefab: GameObject
PlayerVisual
+ApplyDefinition(def: PlayerDefinitionSO)
+SetRenderers(body, head)
PitchAutoAlign
+pitchPlane: Transform
+unityPlaneSize: float
+AlignNow()
MetersToWorldMapper
+fieldWidth: float
+fieldLength: float
+originAtCenter: bool
+lengthAlongX: bool
+ToWorld(x: float, y: float) : : Vector3
FieldBoundsGizmo
-OnDrawGizmos()
BestPassEvaluator
+FindBestPass(dto: FrameSnapshotDTO) : : PassResult
FrameSnapshotApplier
+frameJson: TextAsset
+mapper: MetersToWorldMapper
+runtimeRoot: Transform
-_players: Dictionary
+ApplySnapshot()
-GetOrCreate(id: int) : : GameObject
-EnsureShooterMarker(player: Transform)
ShotGeometryConfig
+fieldWidth: float
+fieldLength: float
+goalWidth: float
+playerBlockRadius: float
+gkBlockRadius: float
ShotOpennessRunner
+RunAnalysis(dto: FrameSnapshotDTO)
ShotDebugDrawer
+openColor: Color
+blockedColor: Color
FrameSnapshotDTO
+frameIndex: int
+targetGoal: string
+players: PlayerDTO[]
PlayerDTO
*
1
PassResult
ShotOpennessEvaluator
+EvaluateOpenness(dto: FrameSnapshotDTO) : : float
-MergeIntervals(intervals: List)
ShotCameraController
+cam: Camera
+backOffset: float
+height: float
+Align(dto: FrameSnapshotDTO)

3.2 Client

3.2.1 View Package (ui)

3.2.1.1 UploadView

class UploadView
Kullanıcının video dosyası seçme, doğrulama, yükleme ve iş başlatma akışını yöneten arayüz sınıfı.
Attributes
private selectedFiles: File[]
private uploadConfig: UploadConfig
private uploadProgress: Map<String, int>
Methods
+ selectFiles(files: File[]) : void
+ submitUpload(matchMeta: MatchMeta) : String
- validateFile(file: File) : bool

3.2.1.2 SummaryPlayerView

class SummaryPlayerView
Üretilen özet videonun oynatılması, zaman kontrolü ve senkronizasyonunu sağlayan arayüz sınıfı.
Attributes
private summaryVideoUrl: String
private playbackState: PlaybackState
private syncOffsetMs: int
Methods
+ loadSummary(jobId: String) : SummaryMedia

+ controlPlayback(action: PlaybackAction) : void
- applySyncOffset(baseTimeMs: int) : int

3.2.1.3 SimulationView

class SimulationView
Kritik an simülasyonunun görselleştirilmesi, sahne kontrolü ve üst katman overlay yönetimini yapan arayüz sınıfı.
Attributes
private sceneId: String
private cameraMode: CameraMode
private overlayState: OverlayState
Methods
+ loadSimulation(matchId: String, momentId: String) : SimulationData
+ setCameraMode(mode: CameraMode) : void
- updateOverlay(frameTimeMs: int) : void

3.2.1.4 NarrationView

class NarrationView
AI spiker metni ve ses çıktısının görüntülenmesi, oynatılması ve kullanıcı etkileşiminin yönetildiği arayüz sınıfı.
Attributes
private scriptText: String
private voiceProfile: VoiceProfile
private audioQueue: AudioClip[]
Methods

+ requestNarration(eventIds: String[]) : NarrationResult
+ playNarration(mode: PlaybackMode) : void
- normalizeTranscript(text: String) : String

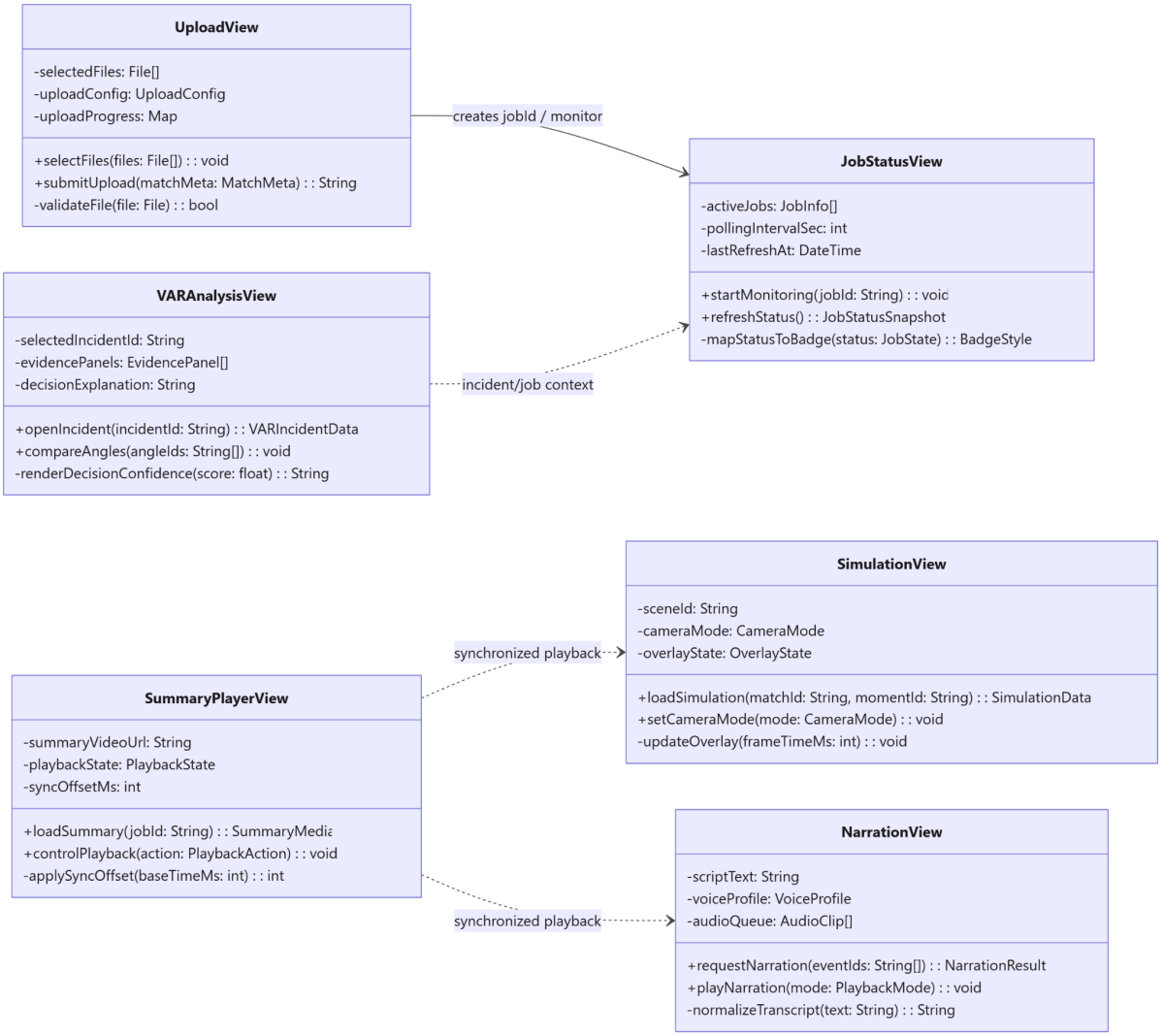
3.2.1.5 VARAnalysisView

class VARAnalysisView
VAR olaylarının kanıt, açı karşılaştırması ve karar açıklamasıyla sunulduğu arayüz sınıfı.
Attributes
private selectedIncidentId: String
private evidencePanels: EvidencePanel[]
private decisionExplanation: String
Methods
+ openIncident(incidentId: String) : VARIncidentData
+ compareAngles(angleIds: String[]) : void
- renderDecisionConfidence(score: float) : String

3.2.1.6 JobStatusView

class JobStatusView
Arka plandaki analiz işlerinin durumunu izleyen, yenileyen ve kullanıcıya bildiren arayüz sınıfı.
Attributes
private activeJobs: JobInfo[]
private pollingIntervalSec: int
private lastRefreshAt: DateTime

Methods
+ startMonitoring(jobId: String) : void
+ refreshStatus() : JobStatusSnapshot
- mapStatusToBadge(status: JobState) : BadgeStyle



3.2.2 Sync Package (sync)

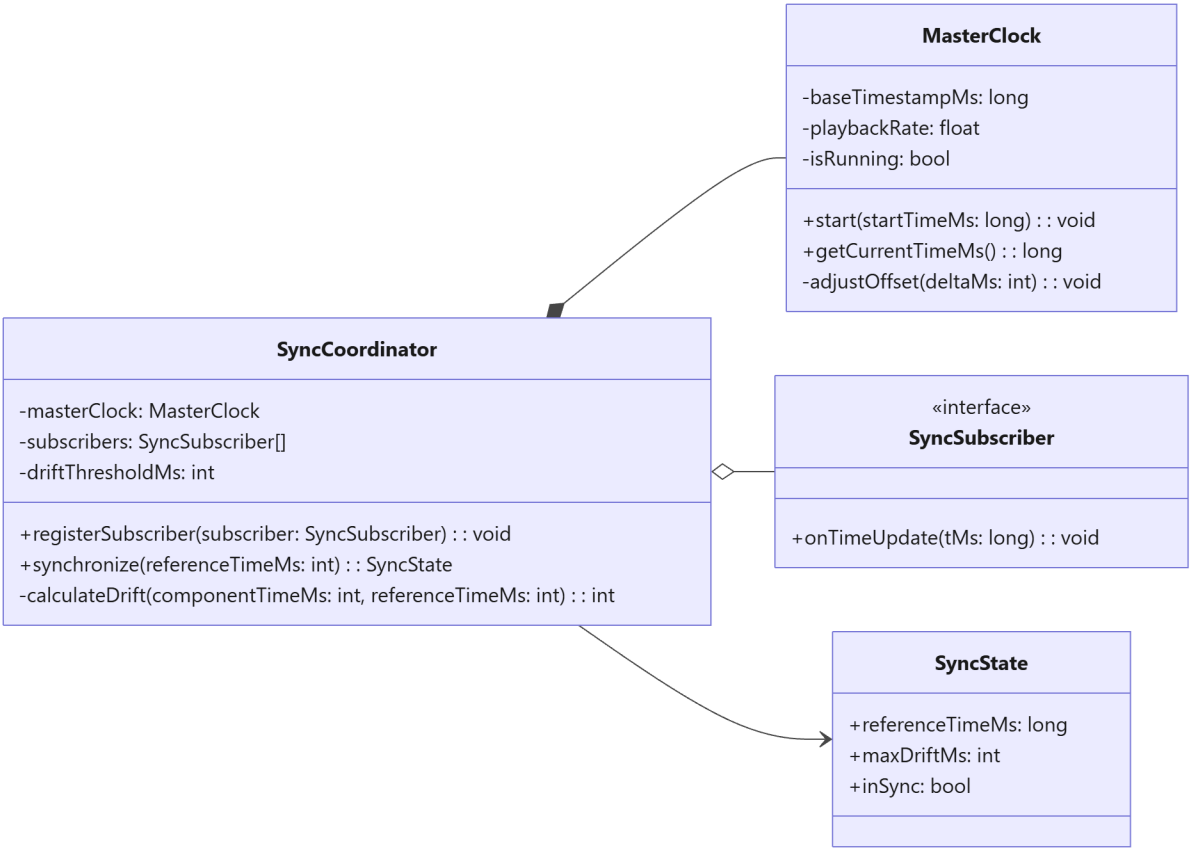
3.2.2.1 SyncCoordinator

class SyncCoordinator
Client tarafındaki medya, simülasyon ve anlatım bileşenlerinin ortak zaman referansına göre senkron çalışmasını yöneten sınıf.
Attributes

private masterClock: MasterClock
private subscribers: SyncSubscriber[]
private driftThresholdMs: int
Methods
+ registerSubscriber(subscriber: SyncSubscriber) : void
+ synchronize(referenceTimeMs: int) : SyncState
- calculateDrift(componentTimeMs: int, referenceTimeMs: int) : int

3.2.2.2 MasterClock

class MasterClock
Client tarafında senkronizasyon için tekil zaman kaynağı sağlayan ve zaman damgası üreten sınıf.
Attributes
private baseTimestampMs: long
private playbackRate: float
private isRunning: bool
Methods
+ start(startTimeMs: long) : void
+ getCurrentTimeMs() : long
- adjustOffset(deltaMs: int) : void

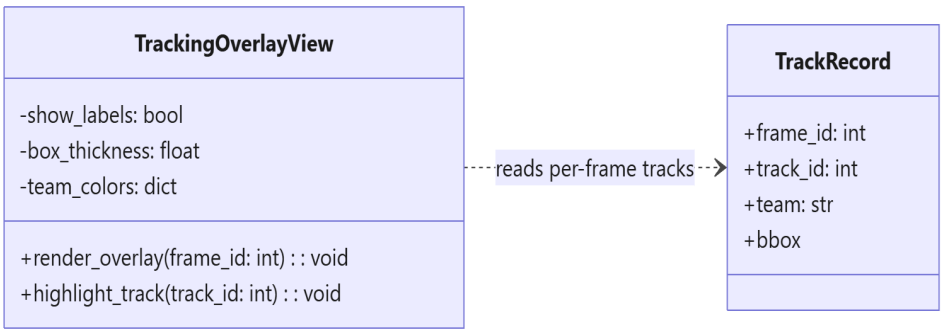


3.2.3 Tracking & Visualization

3.2.3.1 TrackingOverlayView

class TrackingOverlayView
Video oynatıcı katmanı üzerinde, takip edilen nesnelere ait bounding box, takım etiketleri ve track ID bilgilerini dinamik olarak render eder.
Attributes
bool show_labels
float box_thickness
dict team_colors
Methods
+ render_overlay(frame_id : int) : void — TrackRecord verilerini kullanarak mevcut kare için grafik katmanını çizer

+ highlight_track(track_id : int) : void — Kullanıcının seçtiği oyuncunun yörüngesini veya kutusunu görsel olarak vurgular



4. Glossary

Term	Definition
HLD (High-Level Design)	Projenin ana mimarisini, alt sistemlerini ve bileşen ilişkilerini yüksek seviyede tanımlayan doküman.
LLD (Low-Level Design)	Sınıf seviyesinde detayları (attributes, methods, veri akışı, paket yapısı) tanımlayan teknik tasarım dokümanı.
Package	Benzer sorumlulukları olan sınıfların mantıksal olarak gruplanmış hali.
Class Interface	Bir sınıfın dışarı açtığı metot imzaları, veri tipleri ve erişim seviyeleri.
Attribute	Sınıfın durumunu tutan üye değişken.
Method	Sınıfın davranışını tanımlayan fonksiyon.
Public Method (+)	Diğer sınıflar tarafından çağrılabilen metot.
Private Method (-)	Yalnızca sınıf içinde kullanılan metot.
DTO (Data Transfer Object)	Katmanlar arası veri taşımak için kullanılan basit veri yapısı.

API	Bileşenler arası haberleşmeyi sağlayan arayüz sözleşmesi.
Job	Sunucuda asenkron çalışan bir analiz görevi (ör. video işleme).
Job ID	Her analiz işini benzersiz olarak tanımlayan kimlik.
Sync	Farklı medya/katman çıktılarının aynı zaman ekseninde tutulması işlemi.
Drift	İki bileşen arasındaki zaman kayması (ms cinsinden).
MasterClock	Client tarafında tüm senkronizasyonu referans alan merkezi zaman kaynağı.
SyncCoordinator	Client bileşenlerini MasterClock'a göre hizalayan senkronizasyon yöneticisi.
Detection	Model tarafından bir karede bulunan nesne çıktısı (sınıf, kutu, skor).
Tracking	Nesnelerin ardışık kareler boyunca kimlik korunarak izlenmesi.
Homography	Görüntü düzlemi ile saha düzlemi arasında koordinat dönüşümü sağlayan matris.
Event	Analiz sonucunda tespit edilen anlamlı maç aksiyonu (şut, faul, gol vb.).
Possession	Takımların topa sahip olma yüzdesi.
VAR (Video Assistant Referee)	Tartışmalı pozisyonların video ve kural tabanlı analizle yorumlanması süreci.
RAG (Retrieval-Augmented Generation)	Harici bilgi (kural metni vb.) getirme + üretim modelini birlikte kullanan yaklaşım.
Rule Retriever	Pozisyona uygun kural maddelerini veri kaynağından getiren bileşen.
Decision Explainer	Kural ve pozisyon bağlamından açıklanabilir karar metni üreten bileşen.

Vector Store	Metin embedding'lerinin saklandığı ve benzerlik araması yapılan veri katmanı.
Repository	Uygulama verisine erişimi soyutlayan veri erişim sınıfı.
Storage Manager	Video, özet klip, ses ve rapor dosyalarını depolama katmanında yöneten sınıf.
Overlay	Video/simülasyon üstüne bindirilen metin, işaretleyici veya görsel katman.
Playback	Medya içeriğinin oynatma durumu ve zaman kontrolü akışı.
Confidence Score	Model/karar çıktısının güven seviyesini gösteren sayısal değer.

5.Appendix

5.1 Traceability Matrix

Requirement ID	**HLD Component / Service**	**LLD Package / Class**	**Notes**
FR-01	Highlight Detection Service	video_analysis / ObjectDetector	YOLOv8 ile önemli anların (şut, gol) tespiti.
FR-02	Team-Based Association Service	video_analysis / TeamTracker	K-Means clustering ile takım bazlı gruplama (Goal 2).

FR-03	Homography Service	video_analysis / FieldCalibrator	Saha çizgilerinden koordinat dönüşümü.
FR-04	Highlight Compilation Service	content_gen / VideoSummarizer	Tespit edilen anların ffmpeg ile birleştirilmesi.
FR-05	Simulation Generation Service	simulation_engine / ShotOpennessEvaluator	Unity ortamında şut açısı ve gol ihtimali analizi.
FR-06	Simulation Render Service	simulation_engine / ShotDebugDrawer	Konumsal verinin görsel temsile (Wedge mesh) dönüşümü.
FR-07	Rule Retrieval Service (RAG)	var_system / RuleRetriever	IFAB kural kitabından semantik arama ile veri getirme.
FR-08	Decision Explanation Service	var_system / DecisionExplainer	RAG çıktılarını kullanarak kural bazlı yorum üretme.
FR-09	Narration Generation Service	content_gen / NarratorService	Özet ve simülasyonla senkron doğal dilde anlatım.
FR-10	Content Delivery Service	data_layer / MatchRepository	Analiz sonuçlarının API üzerinden sunulması.

FR-11	Synchronized Playback Service	sync / SyncCoordinator	Video, simülasyon ve sesin zaman damgası uyumu.
FR-12	Evidence Packaging Service	var_system / EvidencePackager	VAR kararlarının kanıtlarla birlikte sunuma hazırlanması.

6. References

[1] The International Football Association Board (IFAB), "Laws of the Game 2025/26," Zurich, Switzerland, 2025. [Online]. Available: https://www.theifab.com/

[2] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *2017 IEEE International Conference on Image Processing (ICIP)*, Beijing, China, 2017, pp. 3645–3649.

[3] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[4] G. Jocher *et al.*, "Ultralytics YOLOv8," version 8.0.0, Jan. 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[5] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, UK: Cambridge University Press, 2003.

[6] Object Management Group (OMG), "OMG Unified Modeling Language (OMG UML)," Version 2.5.1, Dec. 2017. [Online]. Availa[7] S. Giancola, M. Amine, T. Dghaily, and B. Ghanem, “SoccerNet: A scalable dataset for action spotting in soccer videos,” in *Proc. IEEE/CVF CVPR Workshops*, 2018. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018_workshops/w34/html/Giancola_SoccerNet_A_Scalable_CVPR_2018_paper.html

[7] A. Deliège *et al.*, “SoccerNet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos,” in *Proc. IEEE/CVF CVPR Workshops*, 2021. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2021W/CVSports/html/Deliege_SoccerNet-v2_A_Dataset_and_Benchmarks_for_Holistic_Understanding_of_Broadcast_CVPRW_2021_paper.html

[8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE CVPR*, 2016, doi: 10.1109/CVPR.2016.91.